

Collections

20.10.2025

Gliederung

- Array (Wiederholung)
- Generics <T>
- Iterable
- Collection
- List
- Set
- Map

Array

```
class ArrayDemo {  
  
    record Student(String name) {}  
  
    void main() {  
        int[] someNumbers = new int[] { 1, 2, 3 };  
  
        Student[] someStudents = new Student[10];  
        someStudents[0] = new Student("Sara");  
        someStudents[1] = new Student("Jeff");  
        someStudents[2] = new Student("Anna");  
  
        for (int i = 0; i < someStudents.length; i++) {  
            IO.println(someNumbers[i]);  
            IO.println(someStudents[i]);  
        }  
    }  
}
```

Definition „Array“ von Oxford:

1. “an impressive display or range of a particular type of thing.”
2. “an ordered series or arrangement.”
3. “an indexed set of related elements.” (computing)

Origin



Middle English (in the senses 'preparedness' and 'place in readiness'): from Old French *arei* (noun), *areer* (verb), based on Latin *ad-* 'towards' + a Germanic base meaning 'prepare'.

Array

```
class ArrayDemo {  
  
    record Student(String name) {}  
  
    void main() {  
        int[] someNumbers = new int[] { 1, 2, 3 };  
  
        Student[] someStudents = new Student[10];  
        someStudents[0] = new Student("Sara");  
        someStudents[1] = new Student("Jeff");  
        someStudents[2] = new Student("Anna");  
  
        for (int i = 0; i < someStudents.length; i++) {  
            IO.println(someNumbers[i]);  
            IO.println(someStudents[i]);  
        }  
    }  
}
```

Ausgabe:

```
1  
Student[name=Sara]  
2  
Student[name=Jeff]  
3  
Student[name=Anna]  
Exception in thread "main"  
    java.lang.ArrayIndexOutOfBoundsException:  
        Index 3 out of bounds for length 3  
        at Demo.main(Demo.java:15)
```

Array

Tue Folgendes nicht!!!!!!!!!!!!!!!!!!!!1

Nutze stattdessen Arrays (oder Collections).

```
class Room {  
  
    private Seat seat1;  
    private Seat seat2;  
    private Seat seat3;  
    private Seat seat4;  
    private Seat seat5;  
    private Seat seat6;  
    private Seat seat7;  
    private Seat seat8;  
    private Seat seat9;  
    private Seat seat10;  
    // ...  
    private Seat seat60000;  
}
```



Generics <T>

Mithilfe von Generics ist es möglich, Klassen, Interfaces und Methoden zu erstellen, die mit verschiedenen Datentypen arbeiten können. Das Ziel von Generics ist es, duplizierten Code zu vermeiden. **Bei der Definition** kann der **Typparameter** (also der Name zwischen den <>) beliebig gewählt werden.

Beim Verwenden des generischen Typs wird der Typparameter (quasi ein Platzhalter) durch ein **Typargument** ersetzt. Als Typargument dürfen allerdings keine primitiven Datentypen wie z. B. int, long, double verwendet werden.

```
class SimpleBag<T> { // Definition: T ist ein Platzhalter für einen Datentyp und wird Typparameter genannt.
    private T item;
    public SimpleBag(T item) {
        this.item = item;
    }
    public T getItem() {
        return this.item;
    }
}

void main() {
    // <Integer> wird Typargument genannt.
    SimpleBag<Integer> bagWithNumber = new SimpleBag<>(42);
    IO.println(bagWithNumber.getItem());
    // <String> wird Typargument genannt.
    SimpleBag<String> bagWithName = new SimpleBag<>("Moin");
    IO.println(bagWithName.getItem());
}
```

Ausgabe:

42
Moin

Generics mit <>

Schreibe niemals Collections ohne spitze Klammern:

Iterable, Collection, Set, List, Map

sondern immer mit spitzen Klammern:

Iterable<>, Collection<>, Set<>, List<>, Map<>

Grund: Ohne spitze Klammern geht Java davon aus, dass die Collection Objekte von Typ Object enthält. Das kompiliert zwar immer, ist aber selten das, was man haben möchte.

```
List listOfObjects = new ArrayList();  
listOfObjects.add(1); // akzeptiert sowohl Integer als auch String  
listOfObjects.add("string"); // akzeptiert sowohl Integer als auch String
```

```
List<Integer> listOfIntegers = new ArrayList<>();  
listOfIntegers.add(1); // akzeptiert nur Integer  
// listOfIntegers.add("string"); // nicht kompilierbar
```

Java Collections Framework

(ein Ausschnitt aus [java.util](https://docs.oracle.com/javase/8/docs/api/java/util/package-summary.html))

Legende:

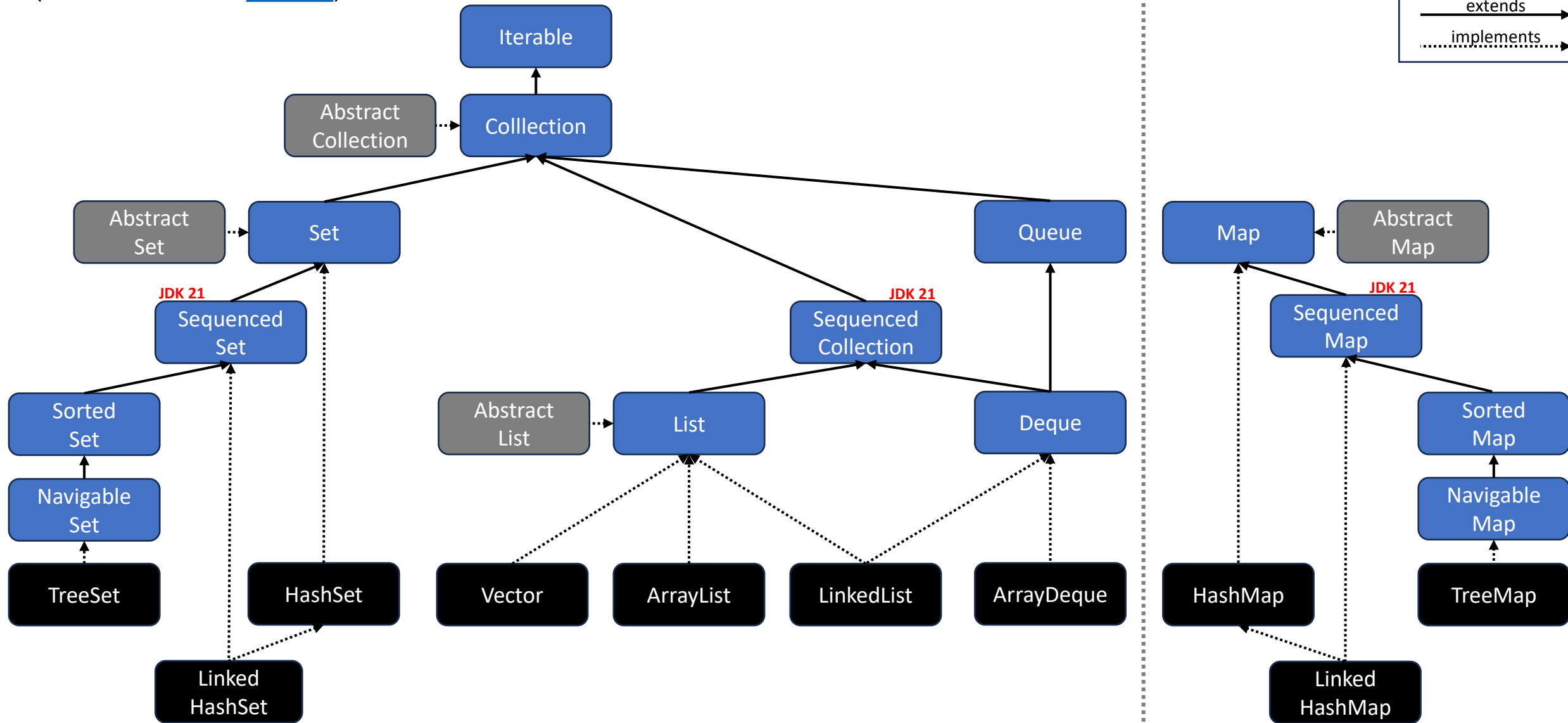
Interface

Klasse

Abstr. Klasse

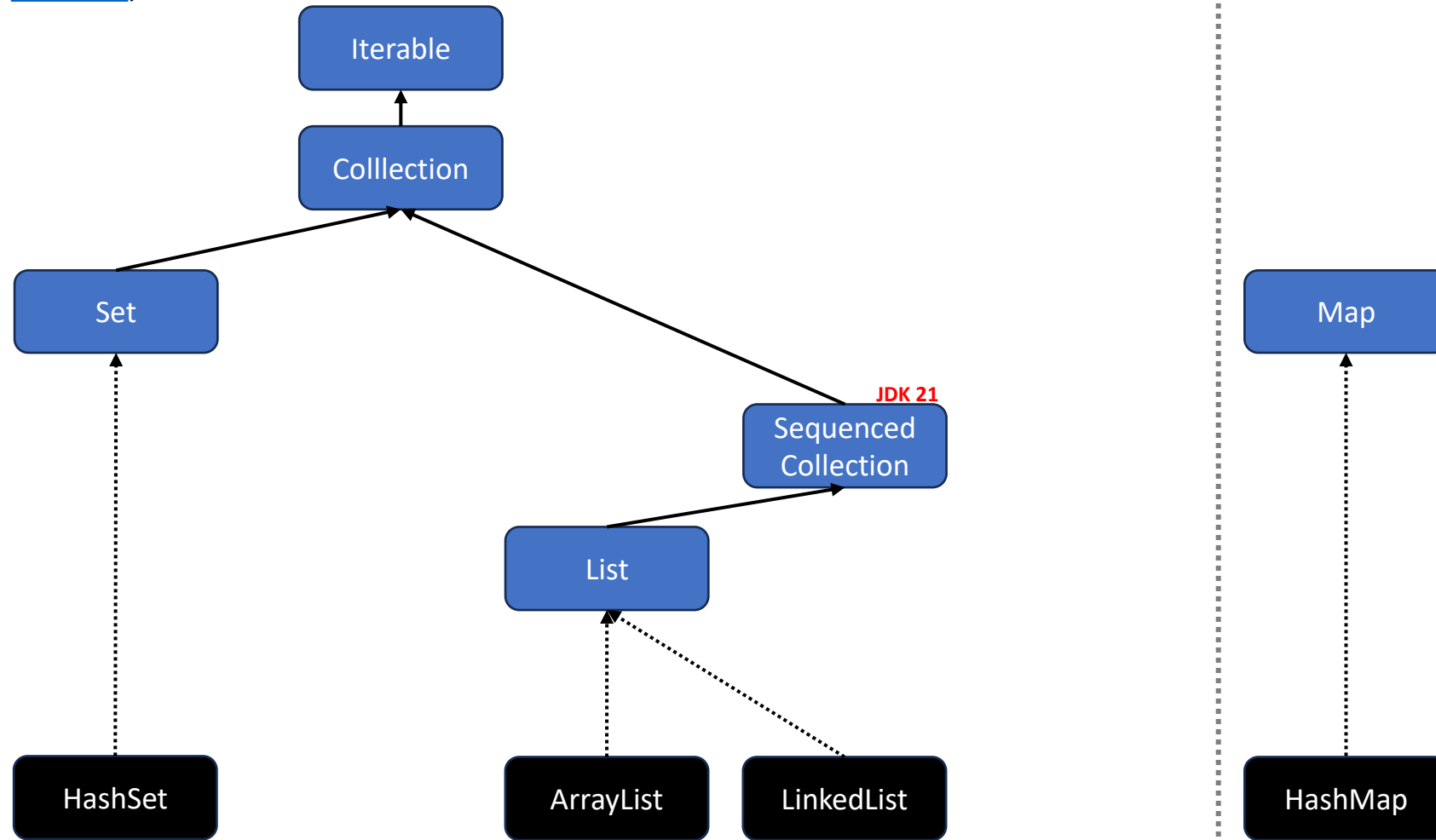
extends

implements



Java Collections Framework

(ein Ausschnitt aus [java.util](https://docs.oracle.com/javase/8/docs/api/java/util/))



Wir schauen uns nur einen Teil an! 😊

Iterable

Definition „iterieren“ von Oxford:

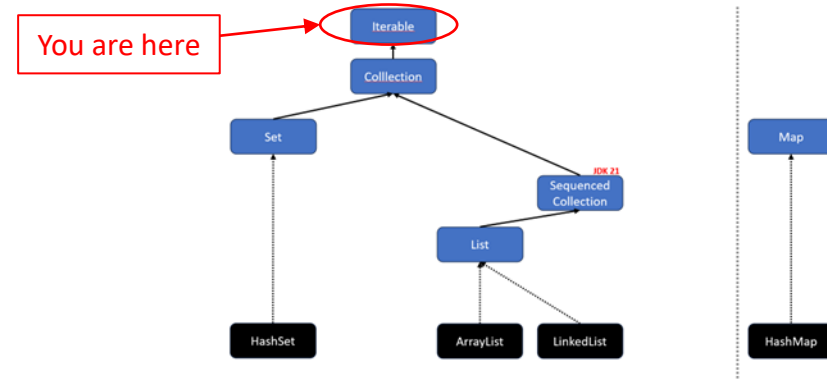
1. (ein Wort, eine Wortgruppe) im Satz wiederholen
2. wiederholen (EDV)

Wenn eine Klasse das Interface Iterable implementiert, kann man ihre Elemente mithilfe eines Iterators durchlaufen. Über die Methode `next()` erhalten wird immer das nächste Element.

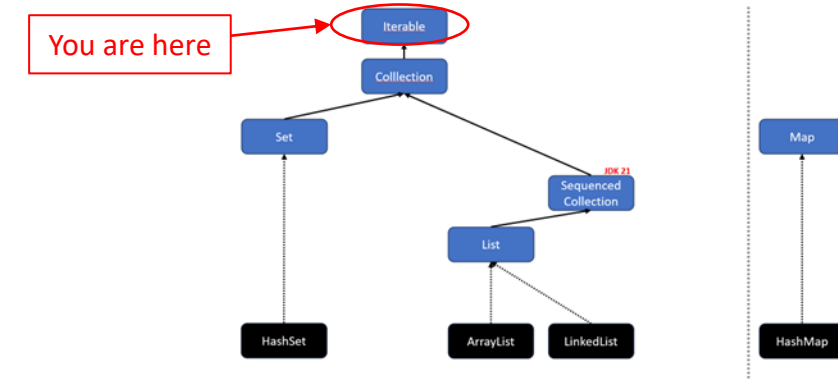
```
void main() {  
    int[] numbers = {0, 1, 2, 3, 4};  
    Iterator iterator = Arrays.stream(numbers).iterator();  
    while (iterator.hasNext()) {  
        Object current = iterator.next();  
        IO.print(current + " ");  
    }  
}
```

Ausgabe:

0 1 2 3 4



Iterable<E>



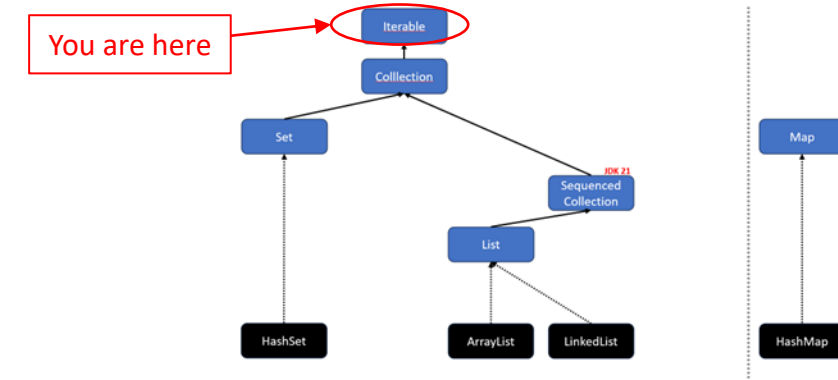
Mithilfe von Generics ist es möglich, ein `Iterable<E>` zu erstellen, das einem bestimmten Datentyp verwendet und zurückgibt.

```
void main() {  
    int[] numbers = {0, 1, 2, 3, 4};  
    Iterator<Integer> iterator = Arrays.stream(numbers).iterator();  
    while (iterator.hasNext()) {  
        int number = iterator.next(); // Java unboxes Integer to int  
        IO.print(number + " ");  
    }  
}
```

Ausgabe:

0 1 2 3 4

Iterable: for-each



Das Interface `Iterable` erlaubt es außerdem, mit dem Java-Schlüsselwort *for* über die Elemente zu iterieren. 🌀

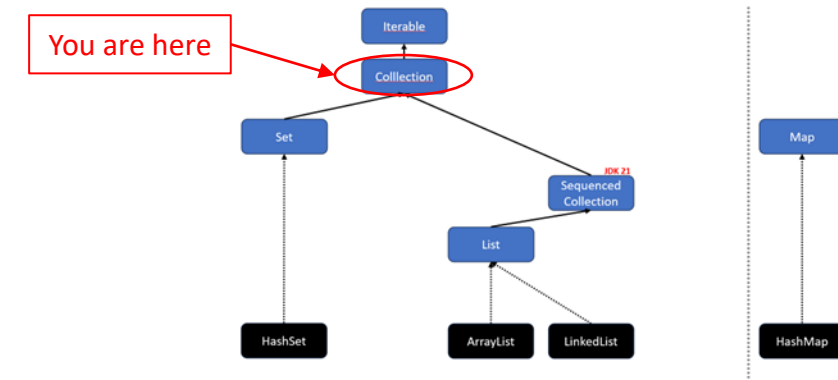
🤪 FYI: *for* funktioniert auch mit Arrays.

```
void main() {  
    int[] numbers = {0, 1, 2, 3, 4};  
    for(int number: numbers) {  
        IO.println(number + " ");  
    }  
}
```

Ausgabe:

0 1 2 3 4

Collection<E>



Das Interface [Collection](#)<E> bietet grundlegende Methoden, um Elemente zu hinzuzufügen oder zu entfernen.

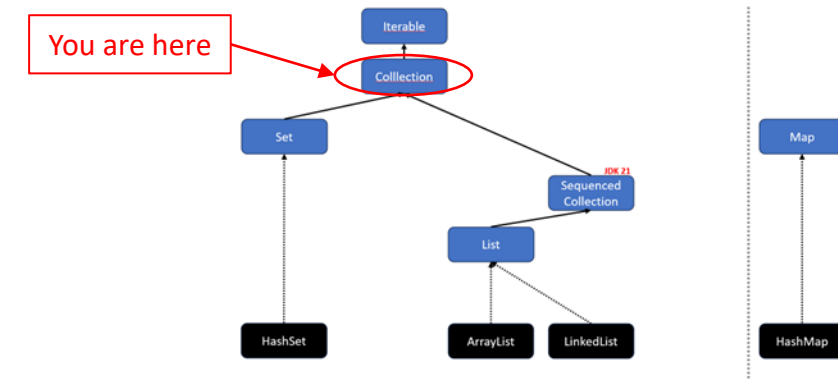
„A collection represents a group of objects, known as its *elements*. Some collections allow duplicate elements and others do not. Some are ordered and others unordered.”

```
boolean add(E e)
boolean addAll(Collection c)    // Vereinigung

boolean remove(Object o)
boolean removeAll(Collection c) // Differenzmenge

boolean retainAll(Collection c) // Schnittmenge
```

Collection<E> – Methoden



Weitere nützliche Methoden des Collection-Interfaces:

// Enthält die Datenstruktur das Element o?

`boolean` `contains(Object o)`

// Gibt die Anzahl der enthaltenen Elemente zurück

`int` `size()`

// true, wenn keine Elemente enthalten sind; sonst false.

`boolean` `isEmpty()`

// Gibt ein Object-Array mit allen Elementen zurück

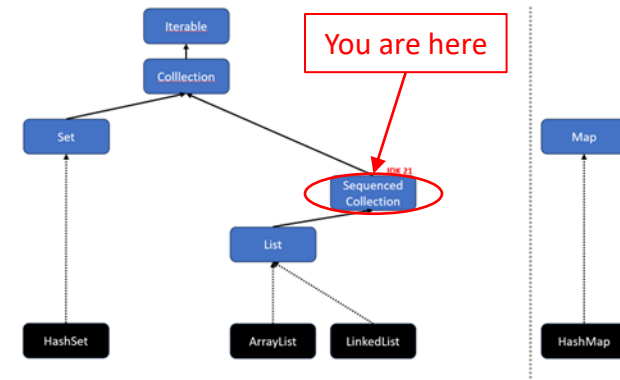
`Object[]` `toArray()`

// Gibt einen Stream mit der Collection als Quelle zurück

`Stream<E>` `stream()` **SPOILER
ALERT!**

SequencedCollection<E>

JDK 21

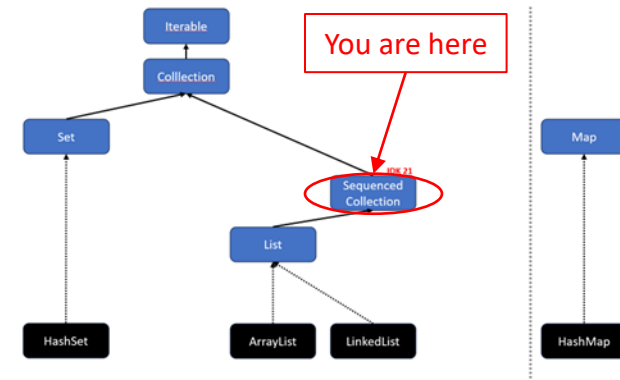


„A collection that has a well-defined encounter order, that supports operations at both ends, and that is reversible. The elements of a sequenced collection have an *encounter order*, where conceptually the elements have a linear arrangement from the first element to the last element. Given any two elements, one element is either before (closer to the first element) or after (closer to the last element) the other element.”

Das Interface [SequencedCollection](#)<E> erweitert das Interface Collection<E> und erlaubt es zudem das erste oder letzte Element zu erhalten oder zu ändern.

<code>void addFirst(E e)</code>	<code>E getFirst()</code>
<code>void addLast(E e)</code>	<code>E getLast()</code>
<code>E removeFirst()</code>	<code>SequencedCollection<E> reversed()</code>
<code>E removeLast()</code>	

SequencedCollection<E>^{JDK 21}

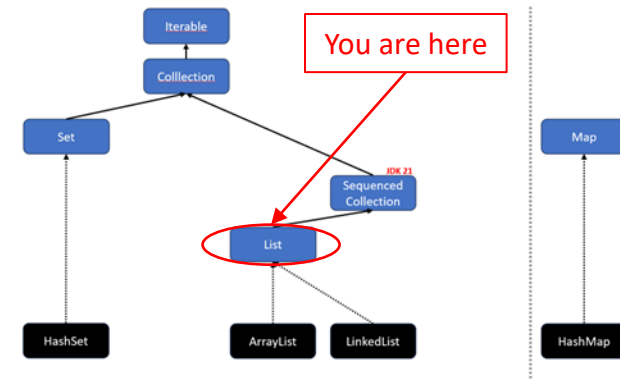


“Motivation: Java’s Collections Framework lacks a collection type that represents a sequence of elements with a defined encounter order. It also lacks a uniform set of operations that apply across such collections. These gaps have been a repeated source of problems and complaints.”

	Get First Element	Get Last Element
List	<code>list.get(0)</code>	<code>list.get(list.size() - 1)</code>
Deque	<code>deque.getFirst()</code>	<code>deque.getLast()</code>
SortedSet	<code>sortedSet.first()</code>	<code>sortedSet.last()</code>
LinkedHashSet	<code>linkedHashSet.iterator().next()</code>	-

Um an das erste oder letzte Element zu kommen, wurde je nach Klasse eine andere Methode angeboten. Das war sehr uneinheitlich. Seit JDK 21 wurde das neue Interface `SequencedCollection<E>` hinzugefügt mit nun einheitlichen Methoden `getFirst()` und `getLast()`.

List<E>



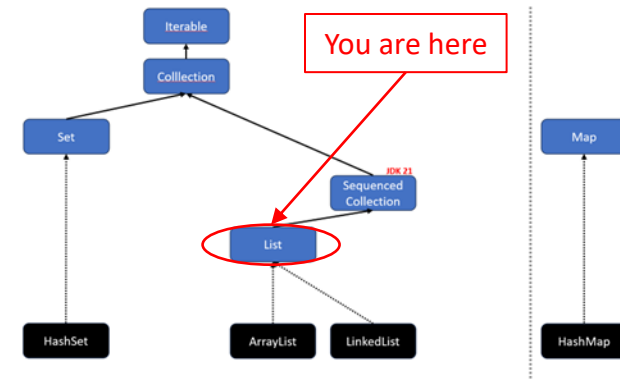
„An ordered collection (also known as a sequence). The user of this interface has precise control over where in the list each element is inserted. The user can access elements by their integer index (position in the list), and search for elements in the list. Unlike sets, lists typically allow duplicate elements.”

Das Interface [List](#)<E> erweitert das Interface `SequencedCollection<E>` und erlaubt zudem auf bestimmte Elemente direkt zuzugreifen z. B. mit `get(index)`.

Häufig genutzte Implementierungen:

- `ArrayList`
- `LinkedList`

List<E> – Methoden



List<E> erweitert SequencedCollection<E> um weitere Methoden:

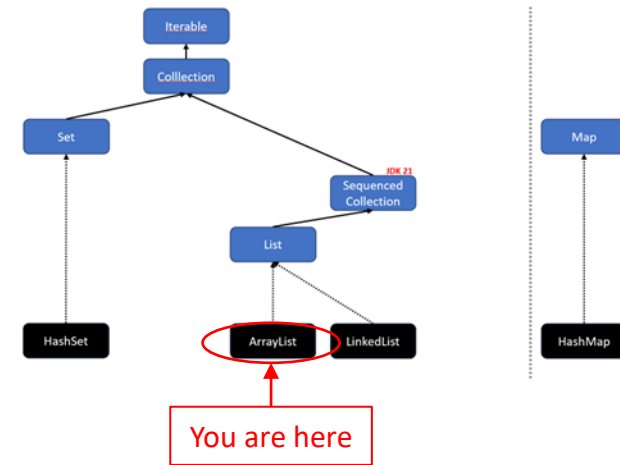
// Gibt das Element an der Position index zurück
E get(int index)

// Fügt das Element an Position index zur Liste hinzu
void add(int index, E element)

// Ersetzt das Element an der Position index und gibt das alte Element zurück
E set(int index, E element)

// Entfernt das Element an der Position index und gibt das entfernte Element zurück
E remove(int index)

ArrayList<E>



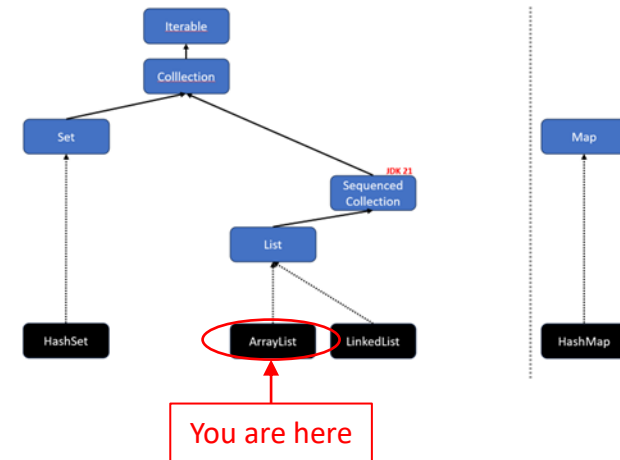
```
void main() {  
    ArrayList<String> names = new ArrayList<>();  
    names.add("Sara");  
    names.add("Alia");  
    for (String name : names) {  
        IO.println(name);  
    }  
}
```

Ausgabe:

Sara
Alia

🧐 FYI: Alternativ kannst du Folgendes schreiben: `var names = new ArrayList<String>();`
Beachte hierbei, dass der Datentyp String auf die rechte Seite gewandert ist,
da Java ihn sonst nicht automatisch ermittelt kann.

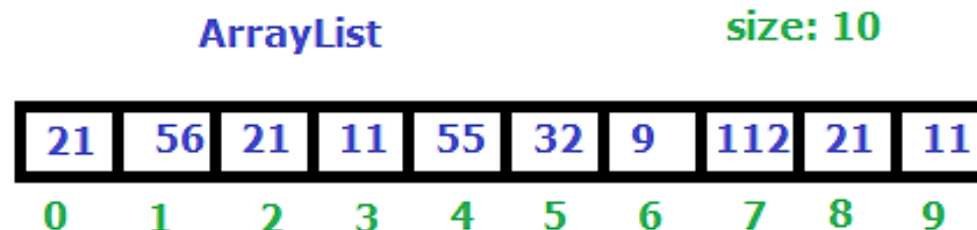
ArrayList<E>



Eine [ArrayList](#)<E> erweitert das Interface `List<E>`. Eine `ArrayList` ist im Grunde wie ein **Array mit variabler Länge** und bietet Methoden, um Elemente zu erhalten oder zu ändern (z. B. `add`, `remove`, `set`, `contains`).

```
ArrayList<Integer> numbers = new ArrayList<Integer>();
```

```
numbers.addAll(List.of(21, 56, 21, 11, 55, 32, 9, 112, 21, 11));
```



Good to know:

Eine `ArrayList` verwendet intern ein gewöhnliches Array, das wenn nötig vergrößert (verdoppelt) wird. Das Vergrößern erfordert allerdings ein kostspieliges Umkopieren aller Elemente in ein größeres Array.

LinkedList<E>

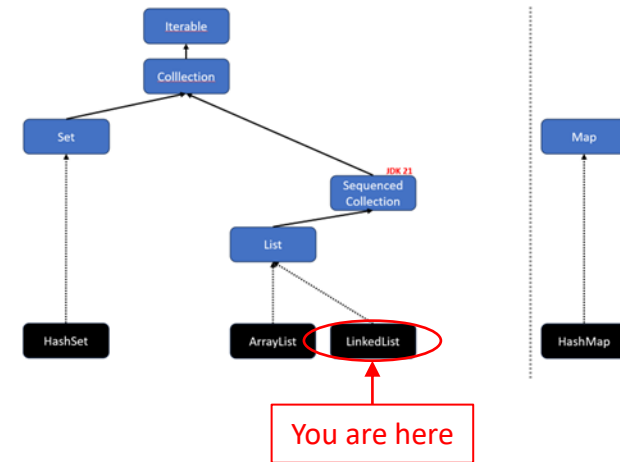
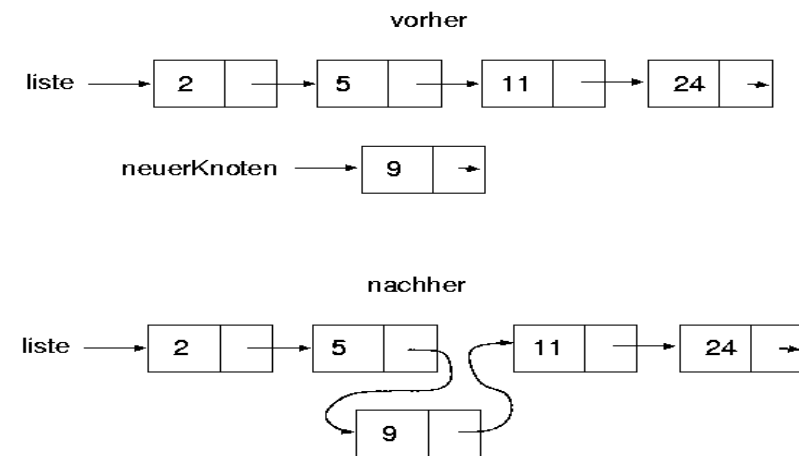
[LinkedList](#)<E> erweitert das Interface List<E> und Deque<E>.

Eine LinkedList ist **optimiert für das schnelle Einfügen und Löschen von Elementen**, indem sie intern Node-Objekte verwendet, die sich referenzieren.

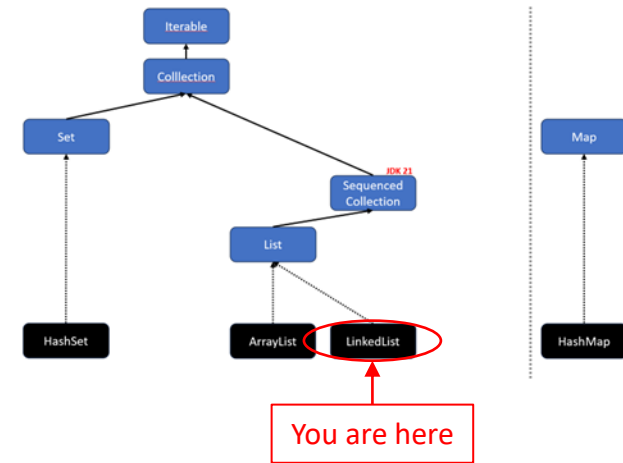
Eine Referenz lässt sich performant durch Zuweisung (nextNode = newNode;) „umleiten“.

So kann beim Vergrößern der LinkedList ein kostspieliges Umkopieren vermieden werden.

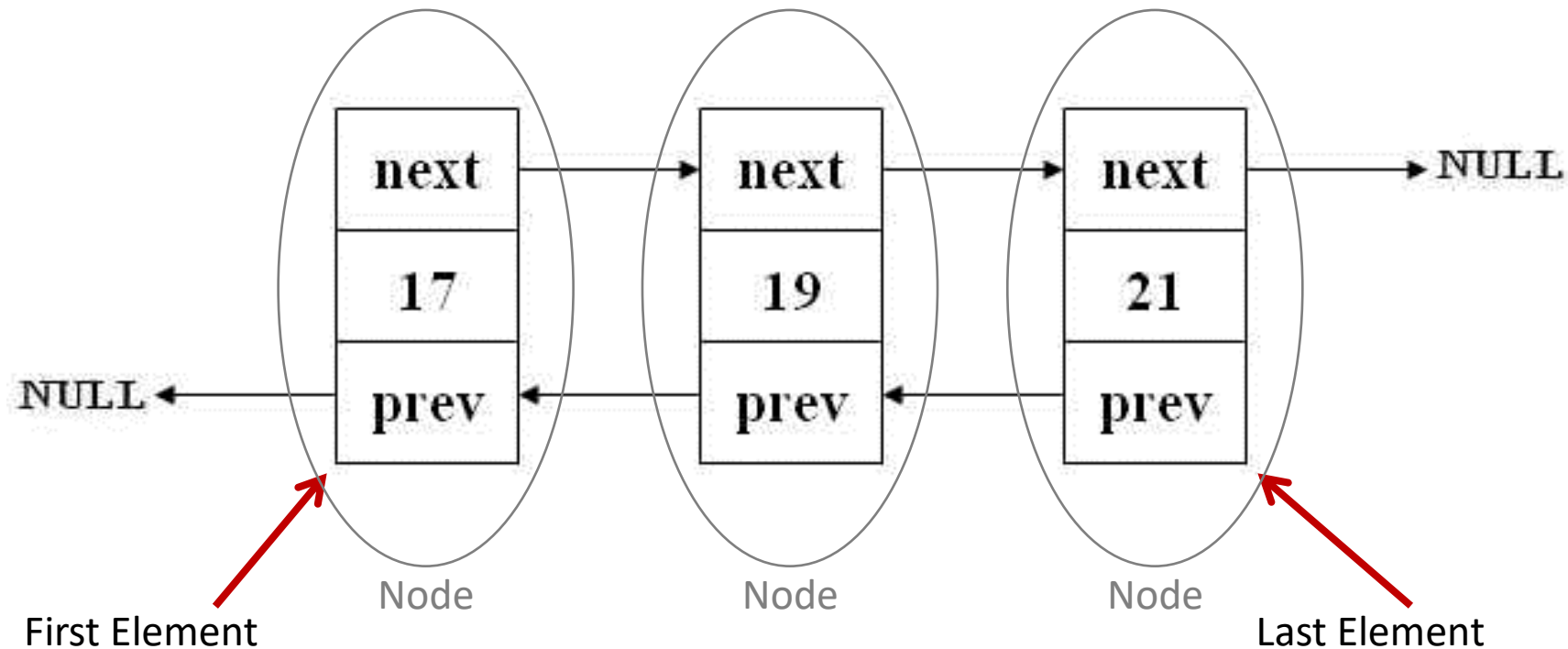
```
LinkedList<Integer> numbers = new LinkedList<>();  
numbers.add(2);  
numbers.add(5);  
numbers.add(11);  
numbers.add(24);  
numbers.add(2, 9); // Füge 9 an Index 2 hinzu.
```



LinkedList<E>



🧐 Java nutzt intern eine doppelt verkettete Liste:



Exkurs: O-Notation

- **O**: order of the function (complexity), or its growth rate.
- **n**: number of inputs, or the length of the array.

Methode	LinkedList	ArrayList
add(E element)	O(1)	O(1), worst case O(n)
add(int index, E element)	O(n), O(1) 1st and last element	O(n)
remove(int index)	O(n)	O(n)
remove(Object obj)	O(n)	O(n)
get(int index)	O(n)	O(1)
contains(Object obj)	O(n)	O(n)



Die [O-Notation](#) wird verwendet, um asymptotisches Verhalten von Funktionen und Zahlenfolgen zu beschreiben. In der theoretischen Informatik wird die Notation genutzt, um Algorithmen zu analysieren und Aussagen über Komplexität, Laufzeit und benötigten Speicherplatz in Abhängigkeit mit der Eingabegröße (Anzahl der Elemente) zu treffen. Es wird dabei immer der „Worst-Case“ betrachtet.

Big-O Complexity Chart

Fakultät!

Beispiel: [Traveling Salesman](#)
(kürzesten Pfad berechnen)

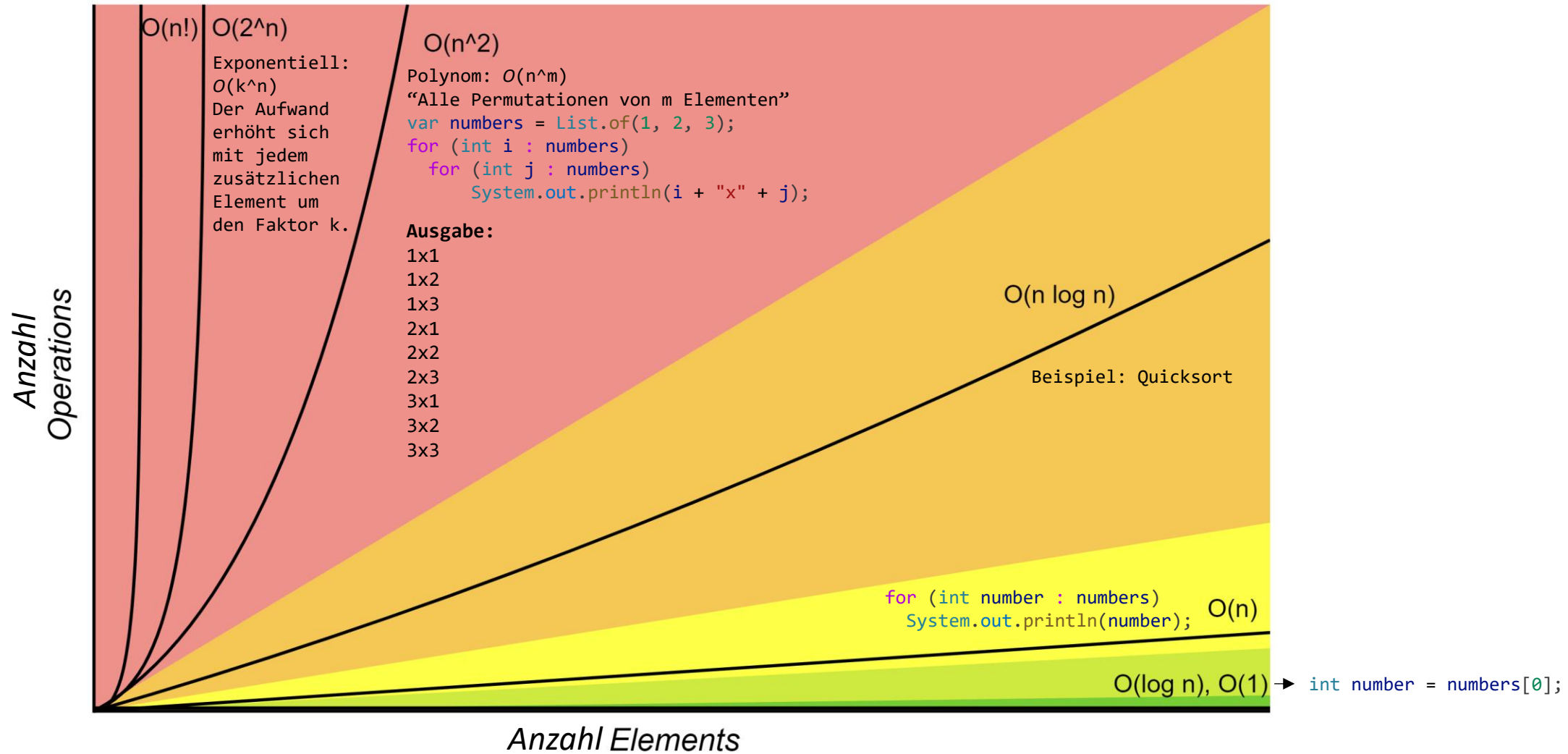
Horrible

Bad

Fair

Good

Excellent



Quelle: <https://www.bigocheatsheet.com/>

Bevorzuge Interfaces

Verwende für Variablen und insbesondere für Parameter Interfaces sofern vorhanden.
Wenn mehrere Interfaces zur Auswahl stehen, verwende das „höchst“-mögliche Interface.

Bei Rückgabewerten empfiehlt sich auch die Nutzung von Interfaces. Dort kommt es aber drauf an, was man anschließend mit dem Rückgabewert machen will bzw. was es dann können soll.

```
public class PreferInterfaces {  
    void main() {  
        Collection<Integer> numbers = new ArrayList<>(); // var numbers = ... ist prinzipiell auch okay.  
        numbers.add(1); // add-Methode aus dem Interface Collection<E>.  
        numbers.add(2);  
        numbers.add(3);  
        IO.println(square(numbers));  
    }  
  
    Iterable<Integer> square(Collection<Integer> numbers) {  
        var result = new ArrayList<Integer>(numbers.size()); // Iterable<E> für den Parameter nicht  
        for (int number : numbers) { // möglich, da size() verwendet wird.  
            int square = number * number; // Je „höher“ das Interface, desto  
            result.add(square); // flexibler kann die Methode verwendet  
        } // werden.  
        return result;  
    }  
}
```

Interfaces: When Java goes wild...

Was passiert, wenn wir hier Collection durch **List** ersetzen? 🤔

```
void main() {  
    Collection<Integer> numbers = new ArrayList<>(List.of(1, 2, 3));  
    IO.println(numbers);  
  
    numbers.remove(1); // Entfernt die Zahl 1 aus der Collection  
    IO.println(numbers);  
}
```

Ausgabe:

```
[1, 2, 3]  
[2, 3]
```

Das Ergebnis von List.of(...) wird dem Konstruktor von ArrayList übergeben. Der Konstruktor fügt alle Elemente in die ArrayList hinzu. Warum ist das Umkopieren in dem Beispiel notwendig?

Interfaces: When Java goes wild...

Wenn wir das Interface Collection zu List ändern, erhalten wir eine unerwartete Überraschung!

```
void main() {  
    List<Integer> numbers = new ArrayList<>(List.of(1, 2, 3));  
    IO.println(numbers);  
  
    numbers.remove(1); // Entfernt die Zahl 1 aus der Collection????????????  
    IO.println(numbers);  
}
```

Ausgabe:

```
[1, 2, 3]  
[1, 3]
```



Interfaces: When Java goes wild...

Polymorphie meets Method-Overloading.

Collection<E> Interface:

`boolean remove(Object o);`

`Collection<Integer> numbers = ...`
`numbers.remove(1);`

List<E> Interface:

`boolean remove(Object o);`

`E remove(int index);`

`List<Integer> numbers = ...`
`numbers.remove(1);`

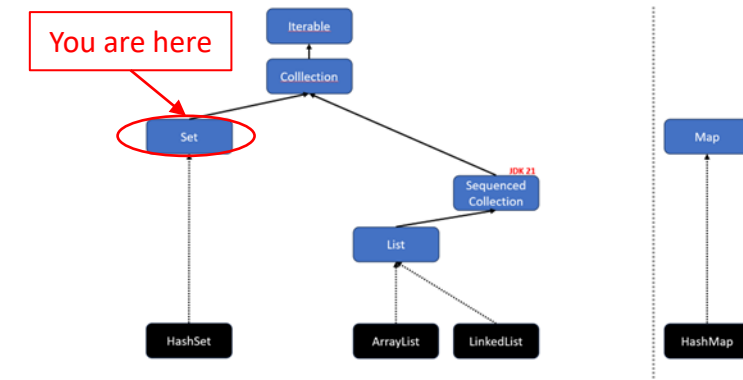


Erklärung: Wenn zwei Methoden denselben Namen und dieselbe Anzahl Parameter haben, verwendet Java die Methode, deren Datentypen direkt übereinstimmen (aka Method-Overloading).

Bei `E remove(int index);` haben die Java-Designer einen Fehler im Interface von List<E> gemacht.

Die Methode zum Entfernen von einem Element an einer bestimmten Stelle (Index) hätte nicht auch remove heißen dürfen sondern z. B. `E removeAt(int index);`. #nobodyisperfect

Set<E>



Ein [Set](#)<E> erweitert das Interface `Collection<E>` (es ist also keine `List<E>`).

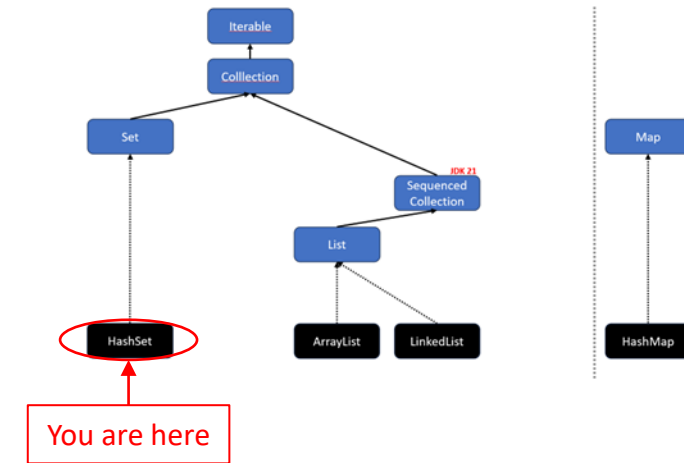
Ein Set entfernt automatisch alle Duplikate und hat keine Ordnung (manchmal).

„A collection that contains no duplicate elements. [...] As implied by its name, this interface models the mathematical *set* abstraction.”

Im Allgemeinen besitzen Sets keine Ordnung. Es gibt allerdings spezielle Implementierungen, die eine Ordnung besitzen:

- `HashSet<E>` Keine Ordnung.
- `LinkedHashSet<E>` Ordnung gemäß der Einfügereihenfolge.
- `TreeSet<E>` Natürliche Ordnung oder eigene Ordnung per [Comparator](#)<T>.

HashSet<E>



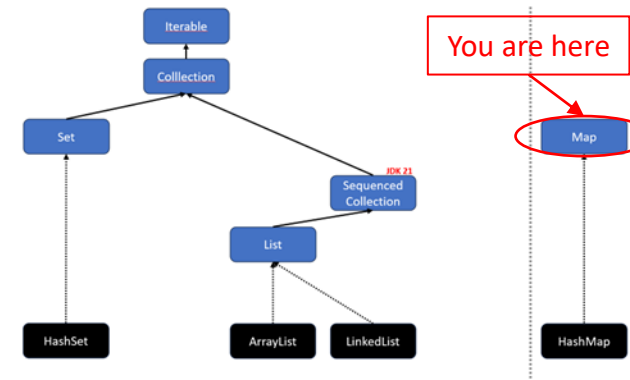
Im Gegensatz zum List-Interface besitzt das Set-Interface keine *get()*-Methode. An die Elemente gelangt man nur mittels Iterator oder for-each.

```
void main() {  
    Set<Integer> uniqueNumbers = new HashSet<>();  
    uniqueNumbers.add(1); // returns true  
    uniqueNumbers.add(2); // returns true  
    uniqueNumbers.add(2); // returns false  
    uniqueNumbers.add(3); // returns true  
    for (int number : uniqueNumbers) {  
        IO.println(number);  
    }  
}
```

Ausgabe:

1
2
3

Map<K,V>



Eine Map<K,V> besteht aus Key-Value-Elementen (bzw. Entry).

Ein Entry<K,V> besteht aus einem Schlüssel (Key) und einem Wert (Value).

Schlüssel dürfen nur einmal in einer Map vorkommen, Werte dagegen beliebig oft.

Man kann sich eine Map wie ein flexibleres Array vorstellen...

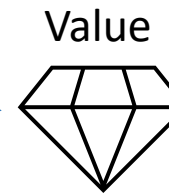
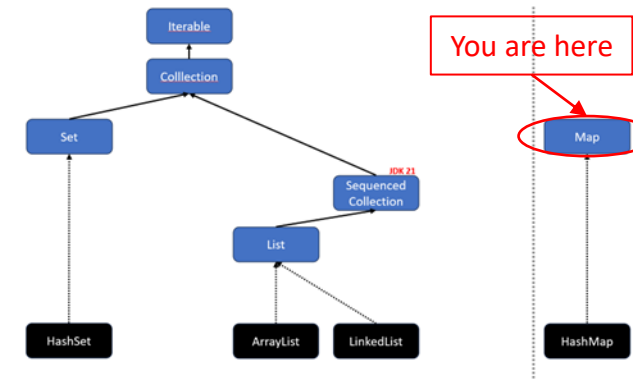
`products.get(1);` VS `products[1];`
map array

...da der Key bei einer Map etwas anderes als ein `int` sein kann (z. B. String).

 FYI: Maps erweitern in Java nicht das Interface `Iterable<E>` oder `Collection<E>`.
Maps sind trotzdem Teil des [Java Collections Frameworks](#).

Map<K,V>

Key: "Fach14"



```
void main() {  
    Map<String, String> cabinet = new HashMap<>();  
    cabinet.put("Fach14", "Diamond");  
    IO.println(cabinet.get("Fach14"));  
}
```

Ausgabe:

Diamond

Map<K,V> – Methoden

// Ein Schlüssel-Wert-Paar hinzufügen
`V put(K key, V value)`

// Prüft, ob der key vorhanden ist
`boolean containsKey(Object key)`

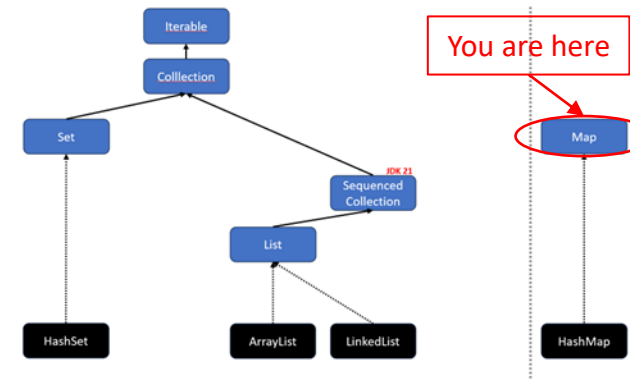
// Einen Wert über seinen Schlüssel bekommen
`V get(Object key)`

// Ein Schlüssel-Wert-Paar entfernen
`V remove(Object key)`

// Ein Set aller Schlüssel der Map erhalten
`Set<K> keySet()`

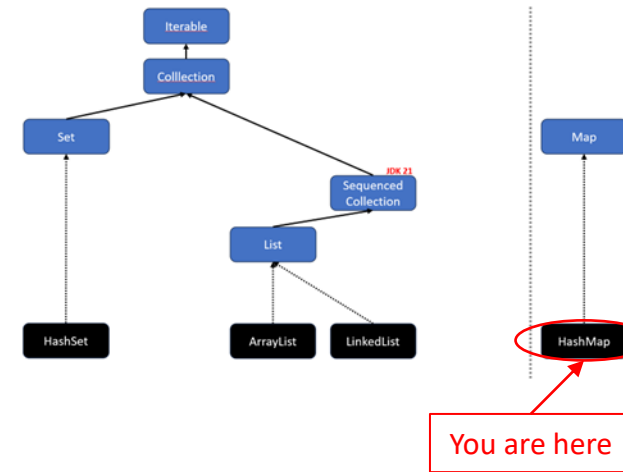
// Eine Collection aller Werte der Map erhalten
`Collection<V> values()`

// Ein Set von Paaren aus Schlüssel und Werten erhalten
`Set<Map.Entry<K, V>> entrySet()`



HashMap<K,V>

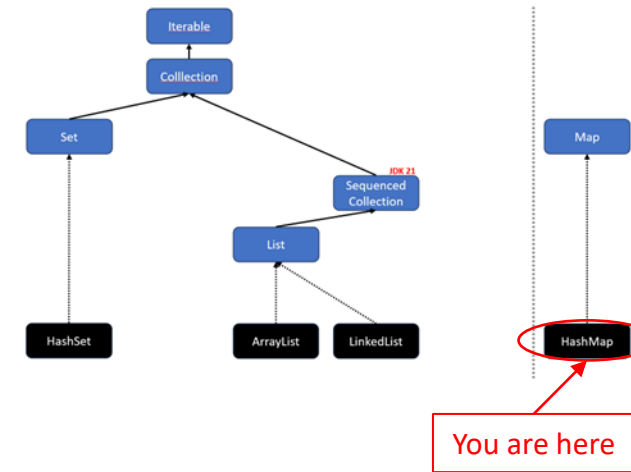
```
void main() {  
    Map<String, LocalDate> projectDeadlines = new HashMap<>();  
  
    projectDeadlines.put("PR2", LocalDate.of(2023, 06, 10));  
    projectDeadlines.put("InfC", LocalDate.of(2023, 06, 12));  
  
    IO.println(projectDeadlines.get("PR2"));  
  
    for (String projectName : projectDeadlines.keySet()) {  
        IO.println(projectName);  
    }  
  
    for (LocalDate deadline : projectDeadlines.values()) {  
        IO.println(deadline);  
    }  
}
```



Ausgabe:

```
2023-06-10  
InfC  
PR2  
2023-06-12  
2023-06-10
```

HashMap – EntrySet<K,V>



```
void main() {  
    Map<String, LocalDate> projectDeadlines = new HashMap<>();  
    projectDeadlines.put("PR2", LocalDate.of(2023, 06, 10));  
    projectDeadlines.put("InfC", LocalDate.of(2023, 06, 12));  
  
    for (Entry<String, LocalDate> projectDeadline : projectDeadlines.entrySet()) {  
        IO.println(projectDeadline.getKey() + "->" + projectDeadline.getValue());  
    }  
}
```

Ausgabe:

```
InfC->2023-06-12  
PR2->2023-06-10
```



FYI: `Entry<String, LocalDate>` kann auch durch `var` ersetzt werden.

(ein Ausschnitt aus [java.util](#))

(ein Ausschnitt aus [java.util](#))



Collections Util-Klasse

Die [Collections](#)-Klasse (beachte das s am Ende) ist eine Utility-Klasse, die hilfreiche statische Methoden für das Arbeiten mit Collections und Maps bereitstellt. Unter anderen sind folgende Methoden enthalten:

- Sortieren (sort)
- Reihenfolge umdrehen (reverse)
- Mischen (shuffle)
- Zwei Elemente tauschen (swap)
- Finden des Minimums oder Maximums (min / max)
- Unveränderliche Collections (z. B. unmodifiableList)
- Thread-Safe Collections (z. B. synchronizedList)



Collections Util-Klasse – Beispiele

```
void main() {  
    List<Integer> numbers = new ArrayList<>(List.of(1, 2, 3, 4, 5));  
  
    Collections.reverse(numbers);  
    IO.println(numbers);  
  
    Collections.swap(numbers, 1, 3);  
    IO.println(numbers);  
  
    Integer min = Collections.min(numbers);  
    IO.println(min);  
}
```

Ausgabe:

[5, 4, 3, 2, 1]

[5, 2, 3, 4, 1]

1

Beachte, dass die meisten Methoden in Collections die übergebene Collection modifiziert, es wird also **keine Kopie bzw. keine neue Collection** erzeugt.

equals und hashCode: contains / containsKey

Die Methode **contains** aus [Collection](#) und **containsKey** aus [Map](#) prüfen, ob ein bestimmtes Objekt enthalten ist (Rückgabe: true oder false).

Bei einer **List** wird für die Prüfung auf Gleichheit die Methode **equals** für jedes Objekt in der Liste aufgerufen.

Bei einem **Set** (z. B. HashSet) verwendet die Methode **contains** zuerst den hashCode() des gesuchten Objekts, um den passenden Bucket zu finden. Danach wird innerhalb dieses Buckets mit **equals** geprüft, ob ein gleiches Objekt vorhanden ist.

Wichtig: Die Methoden **equals** und **hashCode** sollten immer **gemeinsam überschrieben** werden und dieselben Attribute prüfen!

```
class Project {  
    private String name;  
  
    @Override // Überschreibe Default-Methode aus der Klasse Object.  
    public boolean equals(Object object) {  
        return object instanceof Project other &&  
            Objects.equals(this.name, other.name);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hashCode(name);  
    }  
}
```

```
void main() {  
    var projects = List.of(  
        new Project("Train like a Machine"),  
        new Project("RedPill"),  
        new Project("No risk, fun.")  
    );  
  
    var redPill = new Project("RedPill");  
    if (projects.contains(redPill)) {  
        IO.println("No thanks, I take blue!");  
    }  
}
```


equals und hashCode: Dasselbe Attribut

Wichtig: Wenn equals überschrieben wird, **muss** auch die Methode hashCode (aus der Klasse Object) überschrieben werden (und vice-versa).

Beide Methoden, **equals** und **hashCode**, müssen **dieselben Attribute verwenden!**

```
class Project {  
    private String name;  
  
    @Override // Überschreibe Default-Methode aus der Klasse Object.  
    public boolean equals(Object object) {  
        return object instanceof Project other &&  
            Objects.equals(this.name, other.name);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hashCode(name);  
    }  
}
```



Tipp: Die meisten IDEs können equals() und hashCode() automatisch generieren.
#10xDeveloper

equals und hashCode: Dieselben Attribute

Beispiel: Angenommen die Klasse **Project** bekommt ein weiteres Attribut "private String owner;".

Zwei Projekte gelten als gleich, wenn sowohl der Name als auch der Owner gleich sind. D. h., dass in equals und in hashCode beide Attribute verwendet werden müssen!

```
class Project {  
    private String name;  
    private String owner;  
  
    @Override // Überschreibe Default-Methode aus der Klasse Object.  
    public boolean equals(Object object) {  
        return object instanceof Project other &&  
            Objects.equals(this.name, other.name) &&  
            Objects.equals(this.owner, other.owner);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(name, owner);  
    }  
}
```



Ein Hashcode liefert eine ganze Zahl aus allen Attributen, die für die Gleichheit relevant sind.

equals: Regeln

In Java gelten für equals folgende [mathematische Regeln](#), damit die Methode vorhersehbar und korrekt mit anderen Methoden wie z. B. [contains](#) funktioniert:

1. Reflexivität: `x.equals(x) == true`
2. Symmetrie: `x.equals(y) == y.equals(x)`
3. Transitivität: `x.equals(y) && y.equals(z) ⇒ x.equals(z)` // wenn ... und ... dann ...
4. Konsistenz: `x.equals(y)` liefert immer dasselbe Ergebnis, solange Objekte sich nicht ändern.
5. Nicht-null: `x.equals(null) == false`

Verwende **equals** statt **==**, um **Objekte zu vergleichen**.

Ausnahme: primitiven Datentypen.

```
int i = 1;  
int j = 1;  
boolean isEqual = i == j; // vergleiche Primitive mit ==
```



```
String a = "String 1";  
String b = "String 1";  
boolean isEqual = a == b; // funktioniert nicht immer korrekt,  
                           // weil == Speicheradressen vergleicht.  
                           // Verwende equals für Objekte!!!
```



equals und hashCode: Regeln

Der Hashcode hilft dabei, Objekte schnell zu finden oder zu vergleichen, indem er sie in Buckets einsortiert. Wenn du z. B. ein Objekt in eine HashSet oder HashMap einfügst, wird zuerst der Hashcode berechnet, um zu entscheiden, wo das Objekt gespeichert wird. Der Zugriff via Hashcode ist dann beinahe so schnell wie beim Array.

equals und hashCode sollten immer zusammen überschrieben werden. Dabei gilt:

- Wenn zwei Objekte **gleich** sind (laut equals()), **müssen** sie denselben hashCode() haben.
→ Ansonsten kann es passieren, dass zwei logisch gleiche Objekte in einer HashMap- oder HashSet **nicht als gleich erkannt** werden – was zu unerwartetem Verhalten führt.
- Wenn zwei Objekte **verschieden** sind, **können** sie denselben hashCode() haben (Kollision) – das soll möglichst selten vorkommen.

HashSet und hashCode

1. Beim Einfügen eines Objekts (add):

1. HashSet ruft hashCode() auf, um zu bestimmen, **in welchen "Bucket"** das Objekt gehört (je nach Anzahl der Elemente ist ein Bucket eine verkettete Liste oder ein Binärbaum).
2. Innerhalb dieses Buckets prüft HashSet dann mit equals(), ob bereits ein **gleiches Objekt** vorhanden ist.
3. Wenn equals() true ergibt, wird das neue Objekt **nicht eingefügt**.
4. Wenn equals() false ergibt, wird das neue Objekt **im Bucket angehängt**.

2. Beim Suchen eines Objekts (contains):

1. Zuerst wird hashCode() verwendet, um den Bucket zu finden.
2. Mit equals() wird geprüft, ob ein gleiches Objekt im Bucket existiert.

- **hashCode()** ist schnell und liefert eine Zahl → ideal für die schnelle Zuordnung (wie indices) zu Buckets.
- **equals()** ist genauer → wird verwendet, um echte Gleichheit zu prüfen.
- **HashSet** speichert intern die Elemente als Key in einer **HashMap** ab.

→ Durch hashCode ist der Zugriff (get), das Einfügen (add/put) und das Entfernen (remove) sehr schnell $O(1)$, solange es keine Kollisionen bei den Hash-Werten gibt (best case). Gute Verteilung der Hashes ist daher wichtig.

HashMap und hashCode

HashMap speichert intern die Keys als Node-Objekte in einem Array ab, wobei der hashCode des zu speichernden Objekts verwendet wird, um den Index zu berechnen:

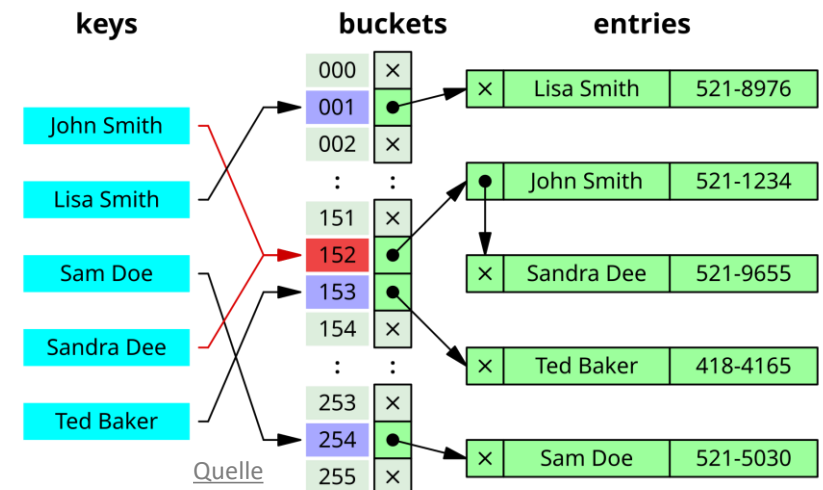
```
int index = hashCode & (array.length-1); // &-Bitmaske ist besonders performant,  
// wenn die Arraylänge eine Zweierpotenz ist.
```

Wenn für mehrere Elemente derselbe Index berechnet wird (Kollision),

- hashCode1 = 17 → binär: 00010001 → index = 00010001 & 00001111 = 00000001 = 1
- hashCode2 = 33 → binär: 00100001 → index = 00100001 & 00001111 = 00000001 = 1

dann werden diese im selben Bucket als Node-Objekte gespeichert. Der Zugriff wird immer langsamer, je mehr Elemente in einem Bucket sind. Bei einer verketteten Liste im Worst Case $O(n)$ und bei einem Binärbaum $O(\log n)$.

Wie bei einer ArrayList wird das interne Array bei HashMap vergrößert (verdoppelt). Dies erfordert ein teures Rehashing und Neuverteilen aller enthaltenen Elemente.



record

Bei **records** werden **equals** und **hashCode** automatisch von Java anhand der Attribute **generiert**, sowie ein **Constructor** mit allen Attributen, die Methode **toString** und **Getter**-Methoden (aber ohne get-Präfix).

Achtung: records sind **immutable** (unveränderlich), daher gibt es **keine Setter**-Methoden!

```
record Project(String name, String owner) { // name und owner sind Attribute.  
    // Java generiert zur Compilezeit automatisch equals und hashCode auf Basis aller Attribute,  
    // sowie den Konstruktor public Project(String name, String owner), die Methode public String toString(),  
    // sowie die Getter public String name() und public String owner().
```

```
void main() {  
    Project deepLearning1 = new Project("Deep Learning 1", "Andre");  
    Project deepLearning2 = new Project("Deep Learning 2", "Andre");  
    IO.println(deepLearning1); // ruft generierte toString()-Methode auf.  
    IO.println(deepLearning1.name() + " mit " + deepLearning1.owner());  
    IO.println(deepLearning2.name() + " mit " + deepLearning2.owner());  
    IO.println();  
    IO.println("dl1.equals(dl2) " + deepLearning1.equals(deepLearning2));  
    IO.println("dl1 == dl2 " + (deepLearning1 == deepLearning2));  
    IO.println();  
    IO.println("DL1 hashCode: " + deepLearning1.hashCode());  
    IO.println("DL2 hashCode: " + deepLearning2.hashCode());  
}
```

Ausgabe:

```
Project[name=Deep Learning 1, owner=Andre]  
Deep Learning 1 mit Andre  
Deep Learning 2 mit Andre
```

```
dl1.equals(dl2) false  
dl1 == dl2 false
```

```
DL1 hashCode: 1200217895  
DL2 hashCode: 1200217926
```

List/Set/Map.of(E...)

List.of(E...), **Set.of(E...)** und **Map.of(E...)** sind praktische **Factory-Methoden**, um schnell eine **immutable Collection** mit Elementen zu **erstellen**.

```
void main() {  
    List<Integer> numbers = List.of(1, 2, 3);  
    List<String> strings = List.of("1", "2", "3");  
    List<Person> persons = List.of(new Person("Sara"), new Person("Jeff"));  
    List<Object> personsAsObject = List.<Object>.of(new Person("Sara"), new Person("Anna"));  
  
    IO.println(numbers);  
    IO.println(strings);  
    IO.println(persons);  
    IO.println(personsAsObject);  
}
```


List/Set/Map.of(E...) – immutable

Achtung: Alle **List/Set/Map.of(E...)**-Methoden geben jeweils eine **unveränderbare** Collection zurück! Methoden wie **add**, **put**, **remove** und **set** werfen **zur Laufzeit** eine **UnsupportedOperationException**, d. h., der Compiler gibt zuvor keinen Fehler aus!

```
void main() {  
    List<Integer> numbers = List.of(1, 2, 3);  
    List<String> strings = List.of("A", "B", "C");  
    List<Person> persons = List.of(new Person("Sara"), new Person("Jeff"));  
    List<Object> personsAsObject = List.<Object>of(new Person("Sara"), new Person("Anna"));  
  
    IO.println(numbers);  
    IO.println(strings);  
    IO.println(persons);  
    IO.println(personsAsObject);  
  
    numbers.add(4);  
}
```

Es gibt keine offizielle Methode, um zu testen, ob eine Collection immutable ist, nur der folgende Workaround funktioniert zuverlässig:

```
try {  
    strings.add("D"); // UnsupportedOperationException bei immutable Listen  
    System.out.println("Liste ist veränderbar.");  
} catch (UnsupportedOperationException e) {  
    System.out.println("Liste ist unveränderbar.");  
}
```

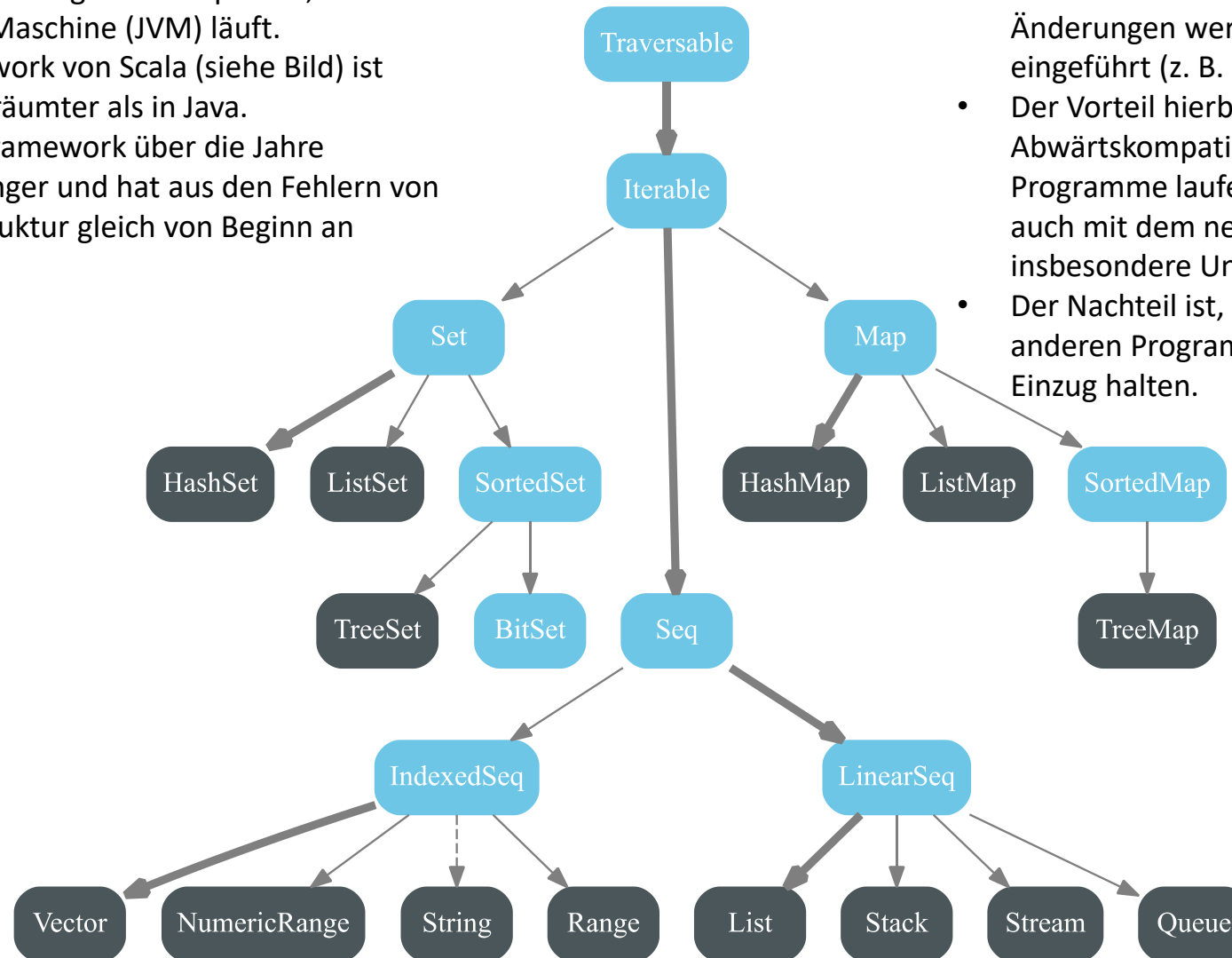
Ausgabe:

```
class java.util.ImmutableCollections$ListN  
class java.util.ImmutableCollections$ListN  
class java.util.ImmutableCollections$List12  
class java.util.ImmutableCollections$List12  
Exception in thread "main" java.lang.UnsupportedOperationException  
    at java.base/java.util.ImmutableCollections.uoe(ImmutableCollections.java:142)
```



Exkurs: Scala Collections Framework

- Scala ist eine funktionale Programmiersprache, die auch auf der Java virtuellen Maschine (JVM) läuft.
- Das Collections Framework von Scala (siehe Bild) ist konsistenter und aufgeräumter als in Java.
- Grund: In Java ist das Framework über die Jahre gewachsen. Scala ist jünger und hat aus den Fehlern von Java gelernt und die Struktur gleich von Beginn an überarbeitet.



- Java ist eine recht konservative Programmiersprache. Änderungen werden nur behutsam und langsam eingeführt (z. B. `SequencedCollection`).
- Der Vorteil hierbei ist, dass Java-Programme eine hohe Abwärtskompatibilität haben, d. h., ältere Java-Programme laufen wahrscheinlich ohne Änderungen auch mit dem neuesten JDKs (sowas mögen insbesondere Unternehmen).
- Der Nachteil ist, dass moderne Sprachelemente aus anderen Programmiersprachen häufig später in Java Einzug halten.